Table des matières

Cours 03 — Charger la RAW 1
Objectifs 1
1. Trois façons d'extraire 2
2. Chargement complet, chargement incrémental 2
3. L'idempotence et le journal des chargements 3
4. La couche RAW 4
5. Déposer sans transformer : le corpus 5
6. Charger une livraison, en pratique 5
À retenir 6
Exercices 6
Annexe A — Le détail des étapes du labo 6

Cours 03 — Charger la RAW

Théorie · Cours 3/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution.

Objectifs

- Distinguer les trois voies d'extraction (fichiers, base, API) et ce que chacune garantit.
- Choisir entre chargement complet et chargement incrémental selon la forme d'une livraison.
- Rendre un chargement idempotent par un journal, et reconnaître ce qu'un journal laisse passer.
- Décrire la forme de la couche `RAW` et justifier chacune de ses règles.
- Déposer tel quel ce qui ne devient pas une table, et rendre ce dépôt idempotent lui aussi.

© 2026 iAlexMG. Tous droits réservés.

1. Trois façons d’extraire

Voie	Chez Boréal	Qui agit	Ce que la source garantit	Ce qui reste à votre charge
Un dépôt de fichiers	les extraits de l’ERP et du portail des garanties, l’archive de la télémétrie, les exports de l’usine	la source écrit, vous ramassez	le fichier, tel qu’écrit	savoir quels fichiers sont nouveaux, et n’en charger aucun deux fois
Une base, par un pilote	la base de l’ERP, dans PostgreSQL	vous interrogez	une réponse cohérente au moment de la lecture	ne pas surcharger la base, et ne lire que ce qui a changé
Une API REST	les observations d’ECCC	vous demandez, page par page	chaque réponse, avec son code HTTP	parcourir toutes les pages, réessayer après une panne, compter ce qui est reçu

- Le dépôt de fichiers est la voie la plus répandue en entreprise, et la plus simple en apparence. Sa difficulté se cache dans le temps : un fichier arrive en retard, arrive deux fois, ou revient corrigé sous le même nom.
- La base et l’API se lisent à la demande ; leur difficulté est de ne rien manquer sans tout relire. Ce cours traite le dépôt de fichiers ; le Cours 04 lit la base et l’API.

2. Chargement complet, chargement incrémental

Forme	Ce que contient chaque livraison	Chez Boréal	Ce qu’on en fait
Complète	toute la table, dans son état du jour	le registre des actifs, la GMAO, l’instantané météo, le catalogue du corpus ; et, chaque mois, les données maîtres de l’ERP (`T005U`, `KNA1`, `MARA`, `MAKT`)	la nouvelle version remplace l’ancienne, ou s’ajoute à côté si l’on veut garder l’historique
Incrémentale	ce qui a été créé ou modifié depuis la livraison précédente	les commandes, leurs postes et les numéros de série de l’ERP ; les extraits des garanties ; les jours de télémétrie	chaque livraison s’ajoute aux précédentes

- Un chargement complet est simple et sûr : on remplace, et l’état est juste. Il coûte cher dès que la table grossit, et il efface la version de la veille.
- Un chargement incrémental coûte peu et garde tout, mais il exige de savoir ce qui est déjà chargé, sans quoi il double.

© 2026 iAlexMG. Tous droits réservés.

- Un extrait incrémental n'est pas une copie de la base. Les extraits de l'ERP comptent 26 538 lignes d'en-têtes pour 25 483 commandes : une commande modifiée le mois suivant revient dans l'extrait suivant, dans sa nouvelle version. L'état de la base se retrouve en gardant, pour chaque commande, la version du lot le plus récent ; l'étape 2 du Labo 04 le fait, à la ligne près.
- Les données maîtres, livrées en entier chaque mois, entrent quand même lot après lot : la `RAW` garde les 49 versions du fichier des concessionnaires, 7 481 lignes pour 156 concessionnaires, de 150 à 156 selon le mois. C'est la mémoire de chaque ouverture et de chaque fermeture, que le remplacement aurait effacée.

3. L'idempotence et le journal des chargements

Un traitement est idempotent quand le relancer donne le même résultat que le lancer une fois. Un chargement se relance tout le temps : après une panne, quand l'orchestrateur rattrape une nuit manquée (Cours 06), quand un collègue veut simplement vérifier.

- Un `INSERT` n'est pas idempotent. Chargez deux fois l'extrait de décembre 2025 : 854 lignes pour 427 commandes, et aucune erreur.
- Trois stratégies protègent un chargement :

Stratégie	Comment	Quand elle convient
Remplacer	vider la cible, puis tout recharger	les livraisons complètes et les petites tables
Fusionner	insérer ou mettre à jour selon une clé (`MERGE`, `INSERT … ON CONFLICT`)	une source qui a une clé fiable, quand on ne veut que l'état courant
Tenir un journal	noter chaque fichier chargé, et ne charger que ceux qu'on ne connaît pas	les livraisons de fichiers, quand on veut garder chaque version

- La `RAW` de Boréal garde chaque version : elle tient un journal, la table `pipeline.chargements`. Pour chaque fichier : son chemin, son empreinte, son lot, ses lignes et l'heure du chargement.
- L'empreinte est un résumé calculé sur le contenu, un SHA-256 : deux fichiers identiques ont la même, un seul octet changé en donne une autre.

Le fichier	Ce que fait le chargeur
inconnu du journal	il le charge et le note
connu, avec la même empreinte	il le saute
connu, avec une autre empreinte	il retire les lignes de l'ancienne version, charge la nouvelle et la note : c'est ainsi qu'un export complet se met à jour

© 2026 iAlexMG. Tous droits réservés.

- Le chargement d'un fichier et sa ligne au journal s'écrivent dans la même transaction. Si le chargeur s'arrête au milieu, ni la table ni le journal ne gardent une moitié de livraison.

Piège — ce que le journal laisse passer. Le journal reconnaît un fichier par son chemin. L'extrait de mars 2023 a été livré deux fois, sous deux noms, `2023-03` et `2023-03_bis` : 609 commandes entrent deux fois, sans erreur. Reconnaître un fichier par sa seule empreinte attraperait ce doublon, mais les 49 fichiers `T005U` des extraits ont exactement le même contenu : seul le premier serait chargé, et qui prend le lot le plus récent ne trouverait plus aucune région. Le doublon reste donc dans la `RAW`, marqué par son lot, et un contrat de données le signale (Cours 11).

Pourquoi — « au moins une fois », puis idempotent. Aucun transport ne garantit à lui seul qu'un fichier ou un message arrive exactement une fois : ou il risque de le perdre, ou il accepte de le doubler. L'ingénierie des données choisit presque toujours de doubler plutôt que de perdre, puis rend chaque étape idempotente. Livraison au moins une fois, traitement idempotent : l'effet est celui d'une livraison unique.

4. La couche `RAW`

La `RAW` est le produit que cette formation livre à l'ingénieur analytique : les fichiers livrés, devenus des tables de l'entrepôt, sans rien perdre ni rien décider à la place de ceux qui connaissent les règles métier.

Règle	**Pourquoi**
une table par fichier livré : `raw.erp_vbak`, `raw.garanties_clients`, `raw.meteo_quotidien`	le consommateur retrouve la source à partir du nom
toutes les colonnes en texte, avec leurs noms tels que livrés	les zéros de tête (`0000200154`) et les dates vides `00000000` survivent ; aucun type n'est deviné (Cours 01)
un objet JSON dans une colonne JSON	la réclamation et le message gardent leur forme, et se lisent par chemin (`message ->> 'sernr'`)
un champ vide devient `NULL`	la convention par défaut de Snowflake et de Databricks : la `RAW` se charge pareil partout
trois colonnes de lignage : `_fichier`, `_lot`, `_charge_le`	chaque ligne dit d'où elle vient et quand elle est arrivée ; on filtre un lot, on rejoue un fichier, on retrouve une anomalie à sa source
ce qui est livré en double reste, marqué par son lot	le brut n'est plus le brut dès qu'on le corrige, et le test d'unicité de l'ingénieur analytique n'aurait plus rien à prouver

- La `RAW` ne décode aucun code, ne dédoublonne rien et ne joint aucune source. Elle garde les 609 commandes de `2023-03_bis`, les 2 303 doublons de la télémétrie (158 146 messages pour 155 843 identifiants) et chaque version d'une réclamation (5 623 objets pour 3 348 réclamations). Tirer l'état courant d'une table est le premier geste du `STAGING` d'Analytics Engineering.

© 2026 iAlexMG. Tous droits réservés.

- Le bronze du Cours 02 et la `RAW` gardent tous deux le brut, pour des lecteurs différents : le bronze range des fichiers dans le lac, en Parquet, pour qui lit des fichiers ; la `RAW` range des tables dans l'entrepôt, pour qui écrit du SQL et du dbt. La télémétrie vit dans les deux. Les points bruts de l'historien et du compteur restent au bronze : leurs séries nettoyées arriveront dans le schéma `argent` au Cours 08.

À retenir — la `RAW` garde aussi ce qu'on a demandé d'effacer. Le client `CL0011163` a demandé l'effacement de ses données le 2025-09-16 : à partir du lot `2025-09`, ses nom, courriel et adresse arrivent vides, mais le lot `2024-01` les contient encore. Une couche qui garde tout garde aussi les données personnelles des livraisons passées. Ce que la Loi 25 exige alors (purger, chiffrer, restreindre l'accès) se décide avec la gouvernance : c'est la matière de la formation Architecture et Gouvernance.

5. Déposer sans transformer : le corpus

- Toute source ne devient pas une table. La documentation de Boréal (manuels, bulletins de service, politiques de garantie, procédures, guides) se dépose telle quelle dans le bronze, avec son arborescence : 35 fichiers, dont 33 documents, le catalogue `index.csv` et la note de l'équipe.
- Seul le catalogue entre aussi dans la `RAW`, `raw.corpus_index`, parce qu'il se lit en SQL.
- Découper les documents en passages et les indexer pour une recherche appartient à la formation AI Engineering.
- Le dépôt doit être idempotent comme le reste : il compare le contenu de chaque fichier à celui déjà déposé, et ne recopie que ce qui a changé. Il compare les contenus, pas les dates.

6. Charger une livraison, en pratique

Avant de déclarer une couche `RAW` livrée, l'ingénieur de données vérifie toujours les mêmes choses :

Question	Ce qu'on vérifie	Chez Boréal (labo)
Que se passe-t-il sans journal ?	un fichier chargé deux fois	854 lignes pour 427 commandes, sans erreur (1.4)
Un seul programme écrit-il à la fois ?	le verrou de l'entrepôt	« L'entrepôt est occupé » (2.2)
Tout est-il entré ?	les fichiers et les lignes du journal	1 572 fichiers, 513 857 lignes, 22 tables (2.4, 3.2, 3.3)
Le chargement se relance-t-il sans doubler ?	un second passage	0 fichier, `à jour` partout (2.5)
Les fichiers vides sont-ils notés ?	les livraisons à 0 ligne	44, 44, 1, 1 et 41 (3.3)
Rien n'a-t-il été deviné ?	le type des colonnes	tout en texte, plus le lignage (3.4)

© 2026 iAlexMG. Tous droits réservés.

Question	Ce qu'on vérifie	Chez Boréal (labo)
Ce qui est livré en double reste-t-il ?	les lots, les identifiants	609 et 609 (3.5) ; 158 146 messages pour 155 843 (3.7) ; 5 623 objets pour 3 348 (3.8)
Que garde-t-on des données personnelles ?	un client effacé, d'un lot à l'autre	`CL0011163`, encore nommé au lot `2024-01` (3.9)
Le dépôt se relance-t-il sans recopier ?	un second dépôt	35 fichiers, puis 0 (4.2, 4.3)

Ces vérifications se font à la main au premier chargement. Le pipeline du Cours 05 les relance à chaque passage, et le Cours 11 en fait des contrats.

À retenir

- Trois voies d'extraction : le dépôt de fichiers, qu'on ramasse ; la base, qu'on interroge par un pilote ; l'API, qu'on tire page par page.
- Un chargement complet remplace, un chargement incrémental ajoute. Un extrait incrémental n'est pas la base : l'état se retrouve en gardant la version la plus récente de chaque clé.
- Un chargement se relance toujours un jour : il doit être idempotent. Un journal des chargements, tenu dans la même transaction que les données, le garantit pour des fichiers.
- Le journal laisse passer un fichier livré deux fois sous deux noms, et c'est voulu : la `RAW` le garde marqué par son lot, et un contrat le signale.
- La `RAW` garde tout, en texte, avec son lignage. Elle ne décode, ne dédoublonne et ne joint rien.
- Ce qui ne se lit pas en SQL se dépose tel quel, et le dépôt compare les contenus, pas les dates.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_03_DE.md`. Faites-les après le labo : plusieurs énoncés supposent que la `RAW` est chargée.

Annexe A — Le détail des étapes du labo

A.1 Étape 1 : l'entrepôt, et un extrait chargé deux fois

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_03_DE.md`, étape 1.

Notions : section 3, l'idempotence ; section 4, la couche `RAW`.

- 1.1. L'entrepôt de Boréal tient en un seul fichier, `plateforme/entrepot/boreal.duckdb`, qu'un seul programme ouvre à la fois. DuckDB crée le fichier, pas son dossier : le même piège que le `COPY` du Cours 02.
- 1.2. `ATTACH` ouvre le fichier dans l'interface, sous le nom `entrepot`, et le crée s'il n'existe pas ; `IF NOT EXISTS` rend la cellule rejouable. Chaque cellule du carnet a sa propre base courante, `memory` par défaut : les requêtes du labo nomment donc chaque table en entier, `entrepot.raw.erp_vbak`.
- 1.3. `all_varchar = true` lit tout en texte, comme la `RAW` le fera ; `filename = true` ajoute le chemin du fichier d'où vient chaque ligne, l'ancêtre de la colonne `_fichier`.
- 1.4. Le `CREATE OR REPLACE` de 1.3 remplace : relancé, il redonne 427. L'`INSERT` ajoute, et ne sait pas que ces lignes sont déjà là. Rien ne plante : c'est ce qui rend le double compte dangereux. Créer une table depuis une requête, la remplacer, y ajouter des lignes : c'est le Cours 03 de la formation SQL avancé, sections 2 et 4 ; ici, seule compte la différence entre remplacer et ajouter.
- 1.5. `DROP SCHEMA … CASCADE` retire le schéma et ses tables d'un coup. L'entrepôt reste attaché, pour montrer le verrou en 2.2.

A.2 Étape 2 : le chargeur

Labo : `Labo_03_DE.md`, étape 2.

Notions : section 1, le dépôt de fichiers ; section 2, complet et incrémental ; section 3, le journal ; section 4, la couche `RAW`.

Ce que fait chaque morceau du chargeur (2.1) :

Morceau	Ce qu'il fait
`PROJET`	le dossier du projet, calculé depuis l'emplacement du script : le chargeur trouve `sources/` d'où qu'on le lance
`LIVRAISONS`	les 22 livraisons : la table de la `RAW`, le motif des fichiers, leur forme, la règle du lot
`LECTEURS`	la requête DuckDB qui lit une liste de fichiers : un CSV tout en texte, un tableau JSON objet par objet, un JSON lines dont le message reste en JSON
`empreinte`	le SHA-256 du contenu : deux fichiers identiques ont la même, un octet changé la change

© 2026 iAlexMG. Tous droits réservés.

Morceau	Ce qu'il fait
`pipeline.chargements`	le journal : chaque fichier chargé, son empreinte, sa table, son lot, ses lignes, l'heure
`a_charger`	les fichiers que le journal ne connaît pas, ou connaît avec une autre empreinte
`BEGIN` … `COMMIT`	une transaction : la table et le journal changent ensemble, ou pas du tout (Cours 03 de la formation SQL avancé, section 8)
`except` … `ROLLBACK`	qui ouvre une transaction la referme ; sans lui, DuckDB la garde avortée, et la requête suivante répond « Current transaction is aborted » au lieu de laisser voir la vraie cause
`DELETE`, puis `INSERT`	un fichier livré de nouveau sous le même nom remplace ses propres lignes
`p.as_posix()`	le chemin s'écrit avec des barres obliques : le lignage est le même sous Windows et sous Linux

- 2.1. Un script SQL suffisait au Cours 02, où chaque bronze se refaisait en entier. Ici, il faut décider fichier par fichier, se souvenir d'un passage à l'autre et tenir une transaction : c'est le travail d'un programme. Le SQL reste dans les requêtes que Python confie à DuckDB, et le Cours 05 fera de ce script un vrai pipeline.
- 2.2. Sous Windows, le programme qui a ouvert `boreal.duckdb` le verrouille : aucun autre ne peut l'ouvrir, même pour le lire. Le chargeur attrape l'erreur de DuckDB et le dit en clair ; la ligne `File is already open in` nomme le coupable, ici `duckdb.exe`, le programme de l'interface.
- 2.3. `DETACH` ferme le fichier : l'interface n'y tient plus.
- 2.4. Les 49 fichiers de chaque table de l'ERP sont les 48 mois et `2023-03_bis` ; les 884 de la télémétrie comptent 41 jours sans message, notés à 0 ligne. La sortie complète suit la liste.
- 2.5. Le journal reconnaît chaque fichier à son chemin et à son empreinte, et n'en charge aucun : c'est l'idempotence qui manquait en 1.4.

La sortie complète de 2.4 :

```
  erp_t005u                  49 fichier(s) chargé(s),      980 lignes
  erp_kna1                   49 fichier(s) chargé(s),    7 481 lignes
  erp_mara                   49 fichier(s) chargé(s),    2 459 lignes
  erp_makt                   49 fichier(s) chargé(s),    4 918 lignes
  erp_vbak                   49 fichier(s) chargé(s),   26 538 lignes
  erp_vbap                   49 fichier(s) chargé(s),   81 292 lignes
```


© 2026 iAlexMG. Tous droits réservés.

```
  erp_cdhdr                    49 fichier(s) chargé(s),       53 lignes
  erp_cdpos                    49 fichier(s) chargé(s),       53 lignes
  erp_zserie                   49 fichier(s) chargé(s),   32 597 lignes
  garanties_clients            48 fichier(s) chargé(s),   25 379 lignes
  garanties_enregistrements    48 fichier(s) chargé(s),   26 732 lignes
  garanties_visites            48 fichier(s) chargé(s),   53 865 lignes
  garanties_consentements      48 fichier(s) chargé(s),   32 708 lignes
  garanties_reclamations       48 fichier(s) chargé(s),    5 623 lignes
  usine_actifs                  1 fichier(s) chargé(s),       23 lignes
  usine_etiquettes              1 fichier(s) chargé(s),       58 lignes
  usine_bons_travail            1 fichier(s) chargé(s),      779 lignes
  meteo_stations                1 fichier(s) chargé(s),       11 lignes
  meteo_quotidien               1 fichier(s) chargé(s),   17 061 lignes
  meteo_horaire                 1 fichier(s) chargé(s),   37 068 lignes
  telemetrie_sorties          884 fichier(s) chargé(s),  158 146 lignes
  corpus_index                  1 fichier(s) chargé(s),       33 lignes
  Total : 1 572 fichier(s), 513 857 lignes, en 12.2 s.
```

A.3 Étape 3 : ce que la `RAW` garde exprès

Labo : `Labo_03_DE.md`, étape 3.

Notions : section 4, la couche `RAW` ; section 3, ce que le journal laisse passer ; section 2, un extrait n'est pas la base.

- 3.1. `READ_ONLY` : on regarde, on ne touche à rien. Sous Windows, un lecteur bloque quand même le chargeur : d'où le `DETACH` de 4.6.
- 3.2. `information_schema.tables` liste les tables de toutes les bases attachées ; `table_catalog` garde celles de l'entrepôt. Sans l'`ATTACH`, la requête rend 0, sans erreur.
- 3.3. Un fichier vide est une livraison comme une autre : le mois sans document de modification donne un fichier d'en-tête seul, le jour sans sortie de véhicule un fichier vide. Le journal les note avec 0 ligne, et un contrôle de volume pourra un jour s'en inquiéter à bon escient.
- 3.4. Aucun type deviné : le chargeur a tout lu en texte. Seul `_charge_le` a un type, l'heure du chargement, que le chargeur écrit lui-même.
- 3.5. Le journal a laissé passer `2023-03_bis`, qui porte un autre nom que `2023-03`. La `RAW` garde les deux, marqués par leur lot : décider lequel compte est une règle métier, et le repérer est le travail d'un contrat (Cours 11).
- 3.6. Plus de lignes que de commandes : les versions successives d'une commande modifiée.
- 3.7. L'opérateur `->>` lit un champ d'une colonne JSON et le rend en texte ; les flèches et le reste de l'extraction sont au Cours 08 de la formation SQL avancé, section 4. Les 2 303 doublons viennent de la livraison « au moins une fois » du courtier MQTT (Cours 09).
- 3.8. Une réclamation revient à chaque changement de statut.

© 2026 iAlexMG. Tous droits réservés.

- 3.9. `prenom IS NOT NULL` montre la présence du prénom sans l'afficher : on explore sans exposer, comme au Labo 01. L'extrait de septembre 2025 arrive sans les données personnelles du client, mais celui de janvier 2024 les contient encore, et la `RAW` le garde.

A.4 Étape 4 : le corpus

Labo : `Labo_03_DE.md`, étape 4.

Notions : section 5, déposer sans transformer.

- 4.1. `rglob("*")` parcourt toute l'arborescence du corpus. `shutil.copyfile` copie le contenu seul : `copy2`, qui copie aussi les dates, échoue quand un conteneur écrit sur un disque Windows, ce que fera l'Airflow du Cours 06.
- 4.2. Le premier dépôt copie tout : 35 fichiers.
- 4.3. Le second compare les empreintes, pas les dates, et ne recopie rien.
- 4.4. Les 35 fichiers sont les 33 documents, le catalogue `index.csv` et la note `LISEZMOI.md` de l'équipe de la documentation. `**` descend dans les sous-dossiers.
- 4.5. Le catalogue est la seule partie du corpus qui se lit en SQL : il est aussi dans la `RAW`.
- 4.6. Le Cours 04 écrit dans l'entrepôt : l'interface doit le lâcher.

A.5 Étape 5 : versionner

Labo : `Labo_03_DE.md`, étape 5.

Notions : aucune section de ce cours ; le dépôt `01_de` et ce qu'il garde viennent du Cours 01, annexe A.1.

- Le code va dans le dépôt. L'entrepôt et le lac vivent dans `plateforme/`, hors du dépôt, et se reconstruisent en relançant les scripts.
- `git add` nomme les deux fichiers : rien d'autre n'entre dans le commit par mégarde.

En entreprise, les mêmes gestes changent d'outil. Les annexes du labo refont le chargement de `VBAK` sur chaque plateforme.

Geste du labo	**Snowflake**	**Databricks**
Déposer les fichiers avant de charger (2)	un stage, interne ou externe	un volume Unity Catalog
Tenir un journal des fichiers chargés (2.4, 2.5)	`COPY INTO` garde 64 jours le nom, la taille et l'empreinte de chaque fichier, et refuse de le recharger ; `FORCE` passe outre et double les lignes ; Snowpipe garde cet historique 14 jours	`COPY INTO` saute les fichiers déjà chargés, même modifiés depuis : un fichier corrigé sous le même nom n'est pas rechargé, sauf avec l'option `force` ;

© 2026 iAlexMG. Tous droits réservés.

Geste du labo	Snowflake	Databricks
		Auto Loader tient ses fichiers traités dans son point de reprise
Lire le chemin du fichier, pour le lignage (2)	la colonne `METADATA$FILENAME`	la colonne cachée `_metadata`
Créer l'entrepôt de la `RAW` (1.2)	une base et un schéma transient, sans sécurité intégrée (*Fail-safe*) : la `RAW` se reconstruit à partir des fichiers	un schéma d'Unity Catalog

© 2026 iAlexMG. Tous droits réservés.